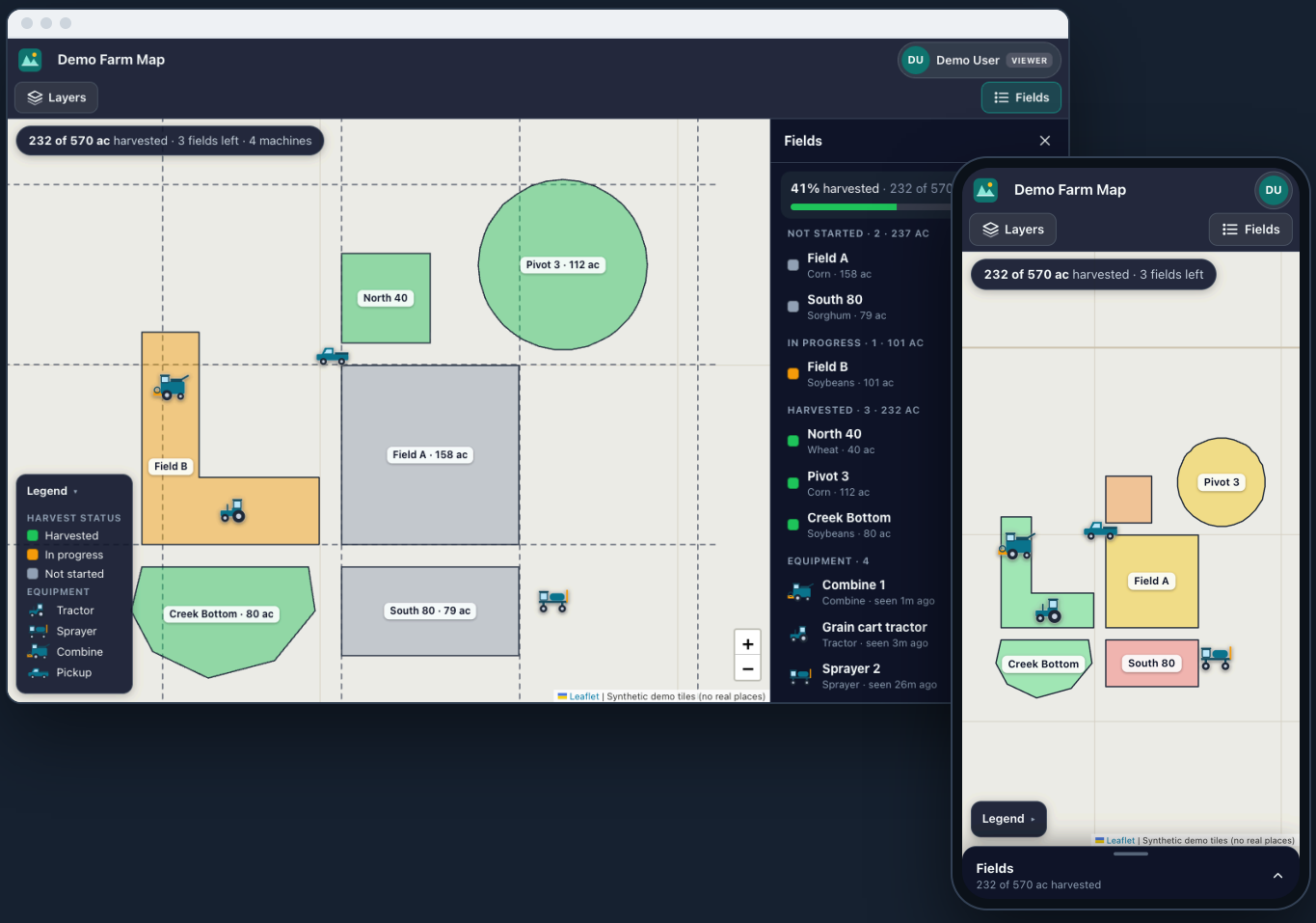
REACT MAPS PLAYBOOK

How our maps work

A friendly tour of the map apps we build: how a map page is put together, where it runs, how it stays safe, and how to build your own with an AI coding agent.



Plain language · Real lessons · Copy-paste prompts · Mobile first

Every map, name and number in this guide is invented demo data.

2026 edition

What's inside

1	**What the maps do** The jobs, and the bones every map shares	3
2	**How a map app fits together** Four parts, one image, and the anatomy of the map page	4
3	**Menus that stay on top** The bug we hit most, and the fix that holds	6
4	**Icons** Status dots, labels, machine clip art, app icons	7
5	**Share links and signed-in maps** Three ways to open one map	8
6	**How data gets onto the map** Downloaded files in, GeoJSON out	9
7	**Where to run it** Your laptop, your own server, or DigitalOcean	10
8	**Keys and scripts** Give the agent scripts, not keys	12
9	**HTTPS with Caddy** One front door with automatic certificates	13
10	**Optional login with Authentik** Who may sign in, logout, passkeys, failing closed	14
11	**Mobile first** Four screen sizes, every time	16
12	**Building it with an AI agent** The loop, the sixteen prompts, a worked example	17
✓	**If you remember nothing else** The short list, and where everything lives	20

How to use this guide

This is the picture book. The playbook's `docs/` folder has the full detail and the code, and `prompts/` has a copy-paste prompt for each piece.

Every chapter ends with two chips: where to read more, and which prompt builds it.

`READ docs/03` `PROMPT 04`

THREE THINGS TO KNOW FIRST

1. **Everything in the pictures is invented.** The demo app in `example/` draws made-up fields. The screenshots use a blank, made-up basemap, so no real place appears. The demo takes a few shortcuts our real apps don't (the README lists them). Our real apps use satellite imagery.
2. **Placeholders stand in for real names:** `example.com`, `203.0.113.10`, `<token>`. Put your own in before you run anything, and use the same placeholders in your own public repos.
3. **You don't have to memorise any of it.** The agent reads the docs. Your job is to know what good looks like, and to check.

WORDS YOU'LL SEE

SPA (single-page app): one web page whose JavaScript draws every screen.

GeoJSON: a text format for shapes (points, lines, polygons), each with a bag of properties.

Portal: React drawing a component somewhere else in the page, usually straight into `<body>`.

Portainer: a web page for running Docker apps ("stacks") on your own server.

Authentik: the identity provider we run ourselves.

Leaflet: the JavaScript library that draws the map. **react-leaflet** lets React use it.

Layer: one set of things on the map (fields, machines, labels) that you switch on or off.

Docker image: the app packed into one file that runs the same everywhere.

Caddy: the web server in front of every app. It handles HTTPS and routing.

Droplet: DigitalOcean's name for a rented virtual server.

Tiles, basemap: the background imagery, loaded as small square pictures.

z-index: the CSS number that decides what's drawn on top.

GHCR: GitHub's container registry, where the images are stored.

OIDC, IdP: the standard behind "Sign in with…", and the server that owns the logins.

Agent: an AI coding assistant that reads, runs and edits code in your repo.

1 What the maps do

We run about a dozen map apps on a working farm. They do different jobs, but they're built from the same handful of ideas.

Field operations
Every field is a shape on satellite imagery, coloured by what was planted, sprayed or harvested, with live machines on top.

Irrigation
Fields coloured by days since they were last watered, and pressure sensors as dots, updated live.

Fleet maps
Sprayers, tractors and pickups as status dots, with the day's track and a replay slider.

Weather
Rain gauges and stations as value badges on a colour scale, with a radar overlay.

Customer portal
Each customer signs in and sees only their own field boundaries.

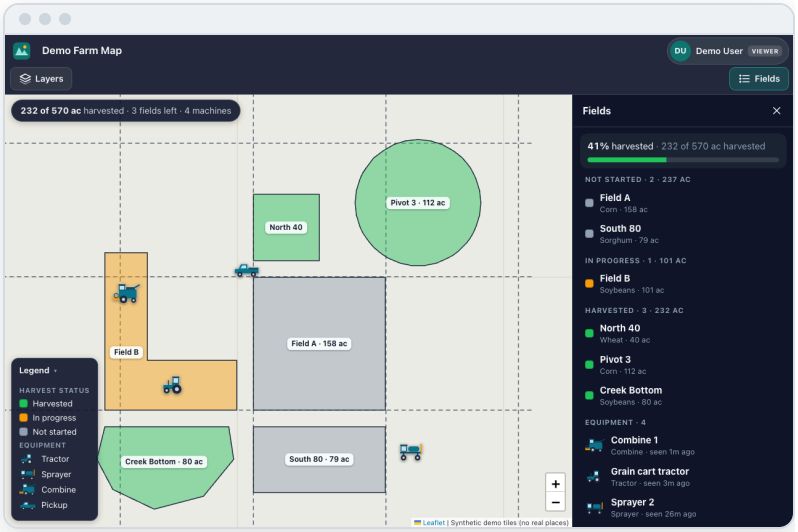
Share links
Someone outside the company opens one map on a phone. No account needed.

Visitor map
A "you are here" dot that keeps working offline, for people walking the farm.

TV kiosk
The same map on a wall screen, hands off, drifting slowly so it never burns in.

Every one of them has the same bones

- **A full-screen map.** The map is the page. Buttons, legends and panels float on top of it.
- **A top bar** with a Home button, the map's name and your user menu.
- **Layers** you switch on and off, and a choice of what colours the shapes.
- **Drawers** that slide out with a list and the details of what you tapped.
- **Popups** with the facts about one field or machine.
- **A phone first.** Most of our maps are used standing in a field.



The demo app in `example/` : invented fields coloured by harvest status, machines on top, a half-mile grid switched on, and the field list open on the right.

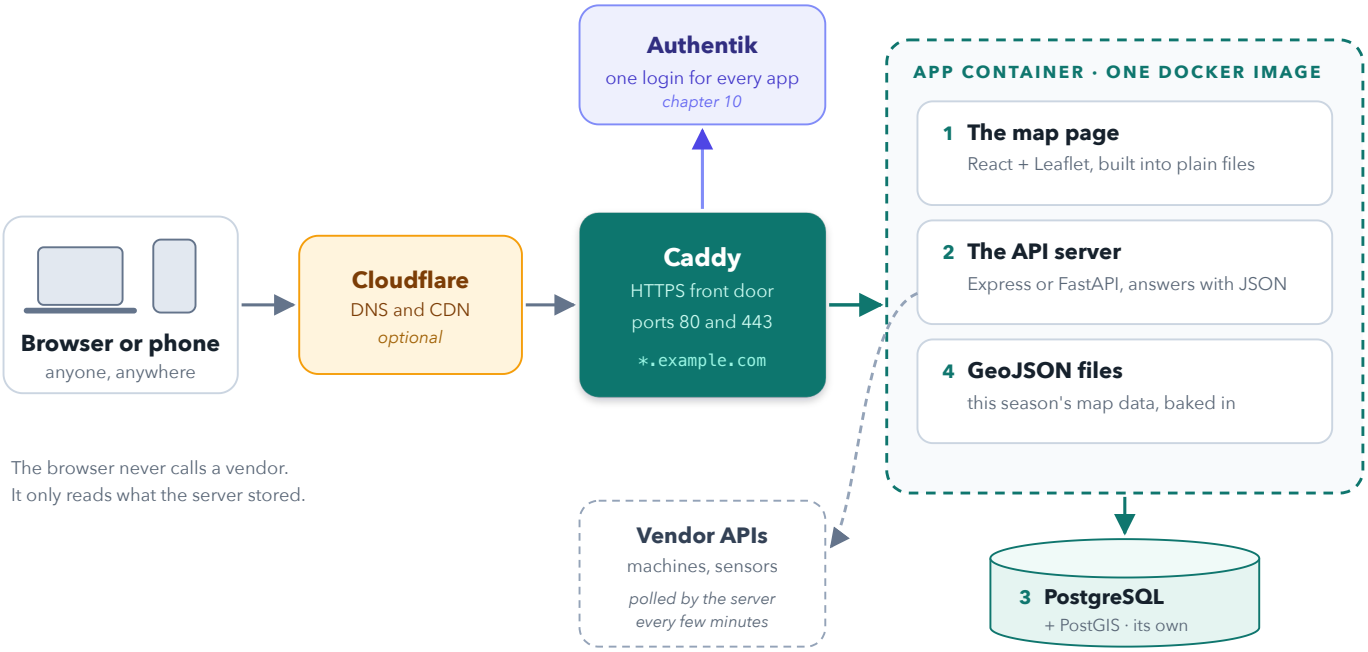
THE STACK IN ONE BREATH

React and TypeScript for the page, Leaflet for the map, a small Express or FastAPI server, PostgreSQL with PostGIS for shapes, static GeoJSON for seasonal data, all packed into a Docker image that GitHub builds. Caddy puts HTTPS in front, and Authentik handles sign-in where it's needed.

`READ README.md` `READ docs/01-architecture.md` `TRY cd example && npm install && npm run dev`

2 How a map app fits together

Four parts, packed into one Docker image, behind one front door.



1. **The map page.** One web page whose JavaScript draws every screen. Leaflet draws the map, and React draws everything around it.
2. **The API server.** Reads the database and answers with JSON. Each route is a short function, with the SQL right there to read.
3. **The database.** PostgreSQL, with PostGIS when there are field shapes ("which field is this point in?", "how many acres?").
4. **Static GeoJSON files.** Map data that changes by season, not by minute. Small tools build it, and it ships inside the image (chapter 6).

Each app is its own small stack

- **Its own repo, its own database.** If app B needs app A's data, app A gets a small API. Two apps never share one database.
- **Two shared services only:** one front door (Caddy, chapter 9) and one login (Authentik, chapter 10).
- **Every app answers `/health` and `/api/about`** without sign-in, so you can always see what's running and which version.
- **A nightly sync is a second container** from the same image, not a cron job hidden on the server.

One image, or two

Shape A: one image start here

The Node server answers the API **and** hands out the built map page. One container, one port. Our field-operations map is built this way.

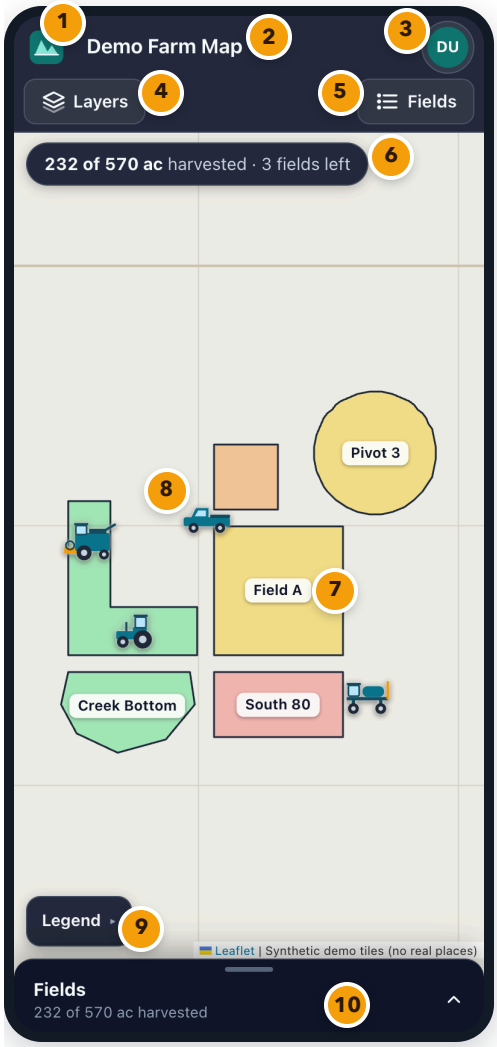
Shape B: split images

nginx hands out the map page and passes `/api` to a separate backend, such as a Python one. Use it when the backend isn't Node, or ships on a different schedule.

`READ docs/01-architecture.md` `PROMPT 01-start-a-new-map-app`

2 Anatomy of the map page

The screen people actually use, from the outside in.



The demo on a 390 × 844 phone.

1. **Home button**
   The app icon. An installed phone app has no Back button, so there's always a visible way home.
2. **Title**
   The map's name, cut short with "…" so the bar never wraps to three lines.
3. **User menu**
   Your initials and role. It opens on top of everything (chapter 3).
4. **Layers**
   Opens the left drawer: switch layers on and off, and pick what colours the fields.
5. **Fields**
   Opens the list sheet: a bottom sheet on a phone (10), a side panel on a laptop. Drawers like Layers slide out the same at every size.
6. **Totals chip**
   The one number people want: 232 of 570 acres harvested.
7. **Shapes and labels**
   One component per layer. Labels are placed so they never overlap and never block a tap.
8. **Machines**
   Last known positions from the equipment feed, drawn as small clip art (chapter 4).
9. **Legend**
   Folds away on small screens.
10. **List sheet**
    Pull it up for the list, grouped by status, with acres per group.

The rules that keep it solid

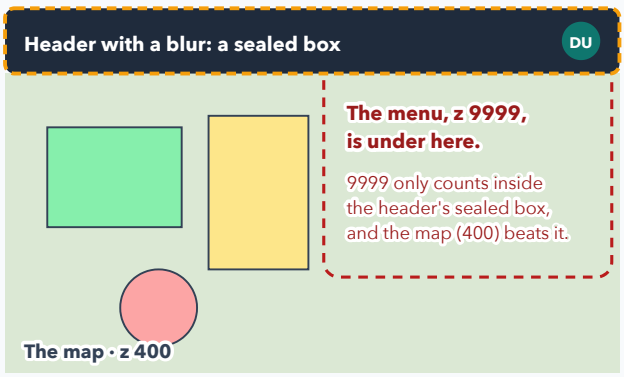
- **Buttons sit beside the map, not inside it.** Anything inside Leaflet's box gets Leaflet's drag and click handling, and stops behaving like a button.
- **Layers live in the address:** `/map?layers=fields,equipment`. Back, refresh and a shared link all reopen the same view.
- **The map stays loaded when you go Home.** Home is drawn over it, so you come back to the same spot and zoom.
- **Leaflet measures its box once.** Tell it when the box changes size (a rotated phone, a drawer opening), or tiles end up off the grid.
- **No blank screens.** A crash shows "Something went wrong / Reload" instead of a white page you can't escape.
- **Every string that becomes map HTML is escaped** with `esc()` first (chapter 4).

`READ docs/02-the-map-page.md` `PROMPT 02-add-a-layer` `PROMPT 03-slide-out-panel`

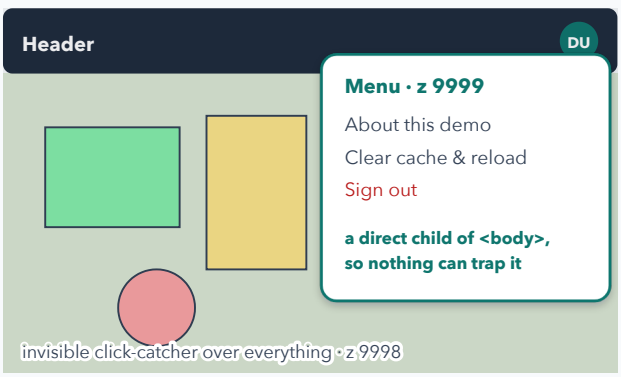
3 Menus that stay on top

The bug we hit most often: the account menu opened behind the map. Three separate apps shipped it, and "just raise the z-index" never stuck.

BEFORE · MENU INSIDE THE HEADER



AFTER · MENU PORTALED TO <BODY>



WHY IT HAPPENS

Leaflet gives its own pieces z-index numbers: tiles 200, shapes 400, markers 600, popups 700, its buttons 1000. A typical "z-50" menu loses to all of them.

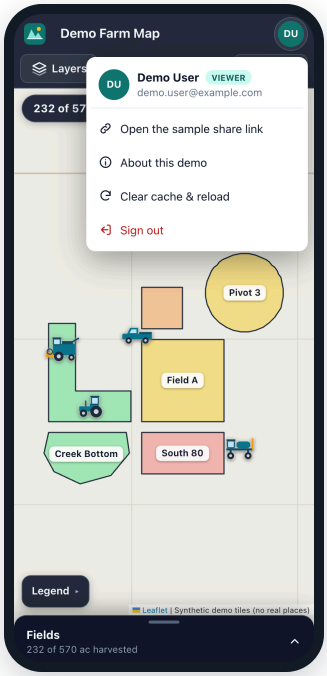
Raising the number doesn't stick, because of **stacking contexts**. A box with a blur, a transform, a sticky or fixed position, or see-through opacity becomes a sealed layer. Anything inside it, even at z-index 9999, can only rise as high as the box itself. A frosted-glass header is exactly that kind of box.

THE RULE

Every dropdown, menu and popover in the app's header is drawn with `createPortal(…, document.body)`, at `position: fixed`, with an invisible click-catcher at z 9998 and the menu at z 9999. **Even a "quick, simple" menu keeps this shell.** Two later apps tried a small local dropdown "just for now", and both shipped invisible.

WRITE THE LADDER DOWN

Z-INDEX	WHAT LIVES THERE
200 – 700	Leaflet: tiles, shapes, markers, tooltips, popups
1000	Leaflet's buttons, the app's top bar, floating buttons
1100 – 1900	Drawers and their backdrops (1500: Home drawn over the map)
2000 – 3000	Dialogs, the crash screen, print preview
9998 / 9999	Click-catcher and menu, **always portaled**



The demo's user menu, open over the map, the drawer button and the totals chip.

DONE MEANS

- It opens on top with a drawer open and a popup open.
- Tapping outside closes it, and that tap does **not** also select the marker underneath.
- Escape closes it. At 320 px wide it fits, with 44 px rows.

`READ docs/03-menus-on-top.md` `PROMPT 04-user-menu-on-top`

4 Icons

None of our apps uses Leaflet's default blue pin. Four kinds of icon do all the work, and all of them are drawn in code.



Tractor Sprayer Combine Pickup

The demo's machine clip art, straight from `example/src/map/equipmentIcons.ts` : one function per kind inside a 64 × 48 box, one palette, every machine facing left. Draw your own; never copy a manufacturer's artwork.



Status dots

Colour carries the meaning: moving, parked, offline. "Offline" is worked out from the last-seen time, and a rough position is dashed.



Badges and labels

Your own HTML on the map. White text gets a dark halo so it reads on any imagery. Labels never catch taps.



Tile icons

Simple 24 × 24 line drawings for the tiles on the Home page, one per map.



App icon set

One piece of artwork, generated by a script into every size: home screen, favicon, browser tab.

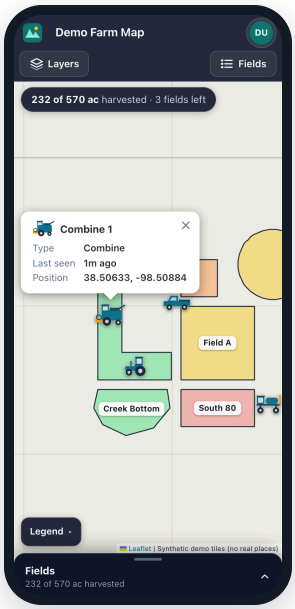
WHICH PICTURE? ONE RULE LIST, MOST SPECIFIC FIRST

Equipment feeds report a model and a category, and the category is often too general. A self-propelled sprayer that calls itself a "Tractor" has to be matched on its model **before** the tractor rule catches it. Keep that rule list in one place, so the markers and the legend always agree.

NEVER SKIP THIS

Escape every string that goes into map HTML. Labels, badges and string popups are set as raw HTML. A field name typed as `<img src=x onerror=…>` would run as code in every visitor's browser. `esc()` turns `< > & " '` into harmless text. Ask for it from day one.

```
export const esc = (t: unknown) =>
  String(t ?? '').replace(/[&<>"']/g, c => ({ '&': '&amp;',
    '<': '&lt;', '>': '&gt;', '"': '&quot;', "'": '&#39;' }[c]!));
```



Tap a machine: its name, type, "last seen 1m ago" and position. The art needs no text label.

`READ docs/04-icons.md` `PROMPT 05-custom-icons`

5 Share links and signed-in maps

Some maps get texted to a contractor or a neighbour with no account. Others are for staff only. So one map can be opened three ways.

`/map?layers=…`
Signed-in staff · everyday work

PAGE
The full app: every layer, drawer and list.

WHAT LETS YOU IN
Your sign-in, checked on every request.

`/m/<id>`
Signed-in staff · special maps

PAGE
The small, fast viewer: an irrigation plan, soil tests, rates with totals.

WHAT LETS YOU IN
A short-lived cookie only the map-data API can read.

`/s/<token>`
Anyone with the link · outsiders

PAGE
The same small viewer. No account, no app data.

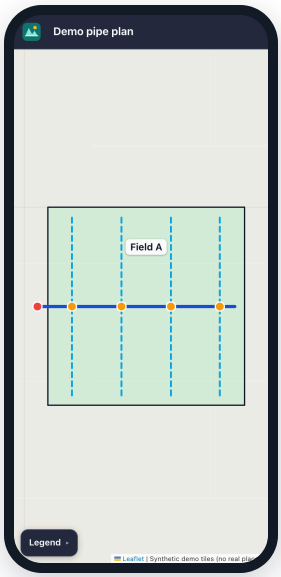
WHAT LETS YOU IN
The link itself. The random token is the only lock.

HOW A SHARE LINK WORKS

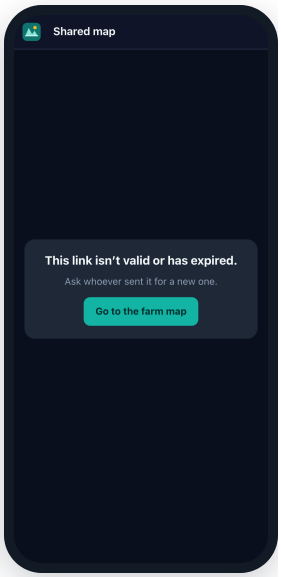
- **The link is `/s/` plus a random token,** such as `pipe-plan-x7k2m9q4ab`. Ten random characters are plenty to stop guessing. Because the token is the lock, **a real one never goes in a public repo.**
- **The token points at one static GeoJSON file.** Public routes never touch the database, and no file path ever comes from the visitor.
- **The viewer page is tiny:** plain TypeScript and Leaflet, no React, no access to the main app. It loads on one bar of signal, and tells search engines to stay away (`noindex`).
- **Several versions behind one link:** "Final | Plan 1 | Plan 2" buttons switch with `?design=`.

YOU WANT TO	DO THIS
Update a shared map	Rebuild its file and redeploy. Same link, new content.
Revoke a link	Delete its entry and redeploy. The link now says it isn't valid.
Replace a leaked link	Add a new token, delete the old one, send the new link.

PUBLIC LINK, PRIVATE EXTRAS
A public link can show the same map staff see, minus the private layers. Signed-in viewers get an extra file appended; a public token never does.



The demo's share link. The demo draws it with React; a real viewer page has none.



A revoked or mistyped link gets plain words.

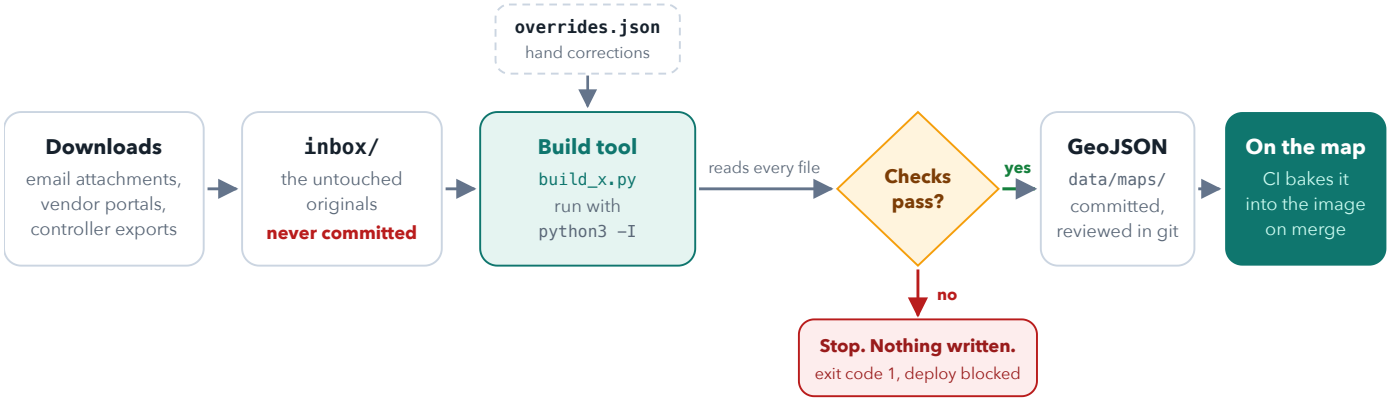
ERRORS IN PLAIN WORDS
No signal: "Check your signal." Not signed in: off to sign in, then straight back. Wrong or old link: "This link isn't valid or has expired." The top bar draws first, so even a failed load has a title.

`READ docs/05-share-links-and-internal-maps.md` `PROMPT 06-public-share-link`

`PROMPT 07-signed-in-only-data`

6 How data gets onto the map

A lot of what our maps show doesn't come from a live feed. It comes from files somebody downloaded. Small Python tools turn them into GeoJSON that ships with the app.



Fast
The server reads a file. No query to plan, nothing to join.

Reviewable
A data change is a git diff you can look at before it ships.

Safe to share
Public share links only ever read static files.

Repeatable
Same inputs, same bytes. A diff shows only real changes.

DOWNLOADED FILES ARE UNTRUSTED

- **Read zips in memory.** Never unzip next to your code, where something could get run.
- **Run tools with `python3 -I`.** Then a planted `json.py` inside a download can't be imported in place of the real one.
- **One broken file never stops the build.** The tool says what it skipped and why, and carries on.
- **Hand corrections are marked.** They live in a committed `overrides.json`, and the output flags them `"set_by_hand": true`.
- **Self-checks compare with the source's own totals.** The tool ends with `All checks OK.` or stops and writes nothing.
- **A public copy refuses to write** if any private text (names, notes, internal IDs) is still in it.

WHAT A FEATURE LOOKS LIKE

```
{ "type": "Feature",
  "properties": {
    "layer": "pipe",
    "name": "Mainline",
    "stroke": "#1d4ed8",
    "stroke_width": 5,
    "note": "Invented demo pipe."
  },
  "geometry": { "type": "LineString",
    "coordinates": [[lng, lat], …] } }
```

The share viewer styles a shape from its properties alone, so a new simple map is often just a new file. Watch the order: GeoJSON is **[lng, lat]**, Leaflet is **[lat, lng]**.

> **LIVE DATA IS DIFFERENT**
> Machines and sensors are polled by the server every few minutes. The browser only reads what the server stored.

`READ docs/06-data-pipeline.md` `PROMPT 08-geojson-build-tool`

7 Where to run it

The same Docker image runs in three places. What changes is who can reach it, who looks after the machine, and what it costs.

Your laptop

WHO CAN REACH IT
You, and your phone on the same Wi-Fi

COST
Free

GOES DOWN WHEN
You close the lid

GOOD FOR
Building and testing

Your own server

WHO CAN REACH IT
Your network, or the internet if you open ports 80 and 443

COST
Hardware and power you already have

GOES DOWN WHEN
The office internet or power drops

GOOD FOR
Internal apps for staff

A DigitalOcean droplet

WHO CAN REACH IT
Anyone on the internet

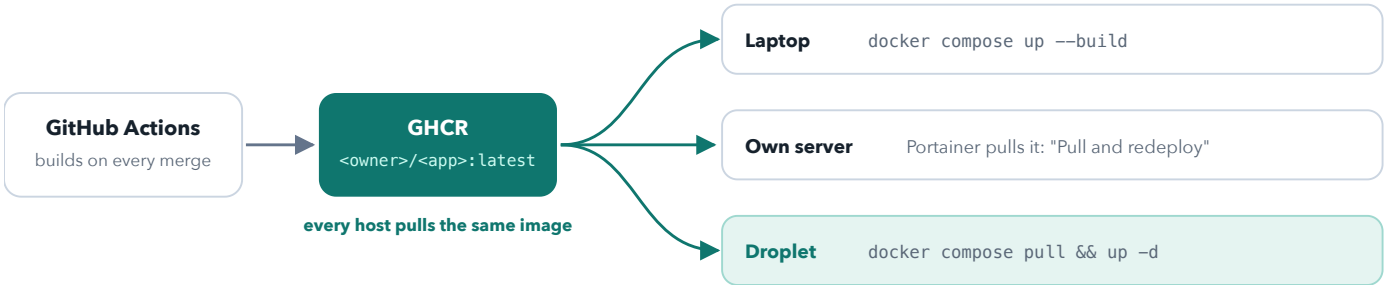
COST
About $6 to $24 a month for a small app

GOES DOWN WHEN
Rarely. It doesn't depend on your building

GOOD FOR
Share links, customer maps, anything outsiders use

A COMMON SPLIT

Internal apps on your own server, behind your login. Anything outsiders use on a droplet, so a power cut at the office can't take a customer's map offline.



ON YOUR LAPTOP

```
$ npm run dev                # localhost:5173
$ npm run dev -- --host      # on your phone too
$ docker compose up --build  # the real image
$ curl -s localhost:8080/health
{"status":"ok"}
```

Before you call a change done, run the **image**, not just the dev server. That catches a broken Dockerfile and files the build left out.

ON YOUR OWN SERVER

- A Linux box with Docker and a fixed address on your network.
- **Portainer** runs each app as a stack straight from its Git repo. The compose file names an image; the server never builds.
- **Caddy** in front for HTTPS (chapter 9), and your **login** for anything private (chapter 10).
- Redeploy from a script with `scripts/portainer/redeploy.py`. It keeps the stack's settings. Done carelessly, a scripted redeploy wipes every secret (chapter 10 has the story).

`READ docs/11-where-to-run-it.md` `READ docs/08-deploy.md` `PROMPT 10-dockerize-ci-portainer`

7 A DigitalOcean droplet in a few minutes

A droplet is a rented Linux computer in someone else's data centre, with its own public address. You log in with an SSH key and run Docker on it, just like your own server.

ONCE, BEFORE YOUR FIRST DROPLET

1. **An account with two-factor login.**
2. **An API token** with custom scopes (droplets, firewalls, reserved IPs, domains, SSH keys, tags) and an expiry, such as 90 days. Save it straight into `secrets/do.env`. **Never paste it into a chat.**
3. **Your laptop's SSH key** on the account. Only the public half (`.pub`) is uploaded.

EVERY TIME

```
$ scripts/do/provision-droplet.sh --dry-run
  the plan and the cost; no token, no API calls
$ scripts/do/provision-droplet.sh
  shows the plan and the price, asks, then does it
droplet_id=123456789
ip=203.0.113.10
$ ssh app@203.0.113.10 cloud-init status --wait
$ scripts/do/check-health.sh \
    https://maps.example.com
HEALTHY: HTTP 200, status ok, …
```

WHAT THE SCRIPT SETS UP

1. Checks that your SSH key is on the account **and** on this laptop.
2. Creates the droplet with a tag, monitoring, backups and the first-boot file.
3. Gives it a **reserved IP**, reusing a spare one before buying another.
4. Attaches the **cloud firewall by tag** (22, 80, 443 in), then checks it really covers the droplet.
5. Creates the DNS record, or tells you exactly which one to add.

Each step is skipped when it's already right, so running it again is safe.

FIRST BOOT DOES THE CHORES

A **cloud-init** file sets the machine up before you ever log in: security updates (and automatic ones after), SSH keys only with no passwords, a firewall and `fail2ban`, Docker with log limits and a weekly clean-up, a non-root user `app`, some swap, and your app behind Caddy. **Nothing secret goes in that file:** anything on the droplet can read it.

WHAT IT COSTS

ITEM	ROUGHLY
`s-1vcpu-1gb`	$6 a month: a static map app like the demo
`s-1vcpu-2gb`	$12 a month: app, API and a small Postgres
`s-2vcpu-4gb`	$24 a month: several apps
Backups	+20% weekly, +30% daily
Reserved IP	Free while assigned, **about $5 a month when it isn't**

Billed by the hour, capped at the monthly price. **A droplet that's switched off still bills.** Only destroying it stops the charge.

TRAPS WE HIT, SO YOU DON'T

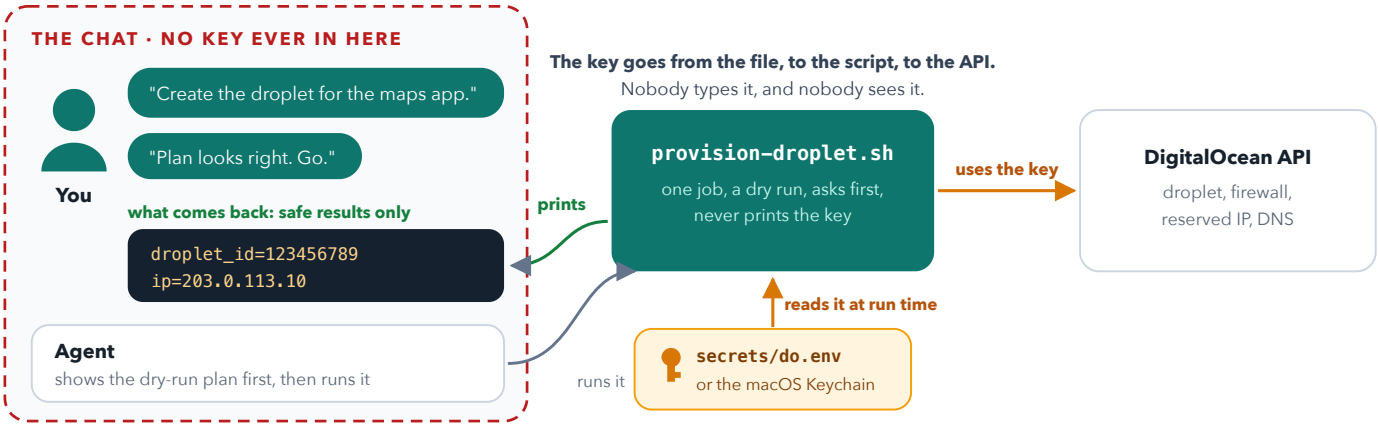
- **A re-created droplet gets a new ID.** A firewall attached by ID protects the dead one. Attach by tag.
- **Destroying a droplet keeps its reserved IP,** still billing. Reuse it or release it.
- **Long jobs over SSH die with the session.** Run them detached with `nohup … &` and watch the log.
- **A 502 is good news, mostly.** DNS, the certificate and Caddy work; the app behind them doesn't.
- **macOS has no `timeout` command,** so a probe built on it always "fails". Use `curl --max-time`.

Tearing down: `destroy-droplet.sh --dry-run`, then the real run, which asks you to type the droplet's name. **DNS kept elsewhere? Delete the A record yourself, straight away.** A record pointing at a released IP can be taken over by whoever gets that IP next.

READ `docs/11-where-to-run-it.md` **FILES** `scripts/do/` **PROMPT** `13-host-on-digitalocean`

8 Keys and scripts

Building with an AI agent gives a key more ways to escape: the chat, the agent's transcript, a settings file, a log, a commit. The fix is simple. Give the agent scripts, not keys.



KEYS LIVE HERE

- ✓ `secrets/*.env` : gitignored as `/secrets/` , `chmod 600`
- ✓ The macOS Keychain
- ✓ On the server: the Portainer stack settings, or the droplet's own `.env`
- ✓ CI secrets in GitHub

NEVER HERE

- ✗ A chat, a prompt, an issue or a screenshot
- ✗ A command line or a URL (shell history, `ps` , logs)
- ✗ An agent's settings file, or any committed file
- ✗ A Docker image, or a server's first-boot file

THE RULES FOR A KEY-SAFE SCRIPT

1. Reads the key when it runs, from `secrets/` or the Keychain. Never takes it as an argument.
2. Never prints it: no `set -x`, no dumping whole API responses.
3. Does **one** job, and its name says which.
4. Prints only safe results: IDs, addresses, names, status codes.
5. Has a `--dry-run` that needs no key at all.
6. Asks before it spends money or touches production.

SCAN BEFORE YOU PUSH

```
$ scripts/secrets/scan-before-push.sh \
    --env-file secrets/do.env \
    --denylist secrets/denylist.txt
  docs/notes.md:14  env-value   value of PORTAINER_URL
  src/config.ts:9   private-ip  private IP (192.168.x)
2 finding(s)  # where and what kind, never the value
```

IF A KEY LEAKS

Revoke it first, then make a replacement, check what the old key did, clean up where it leaked, and fix the cause. A key pasted into a chat counts as leaked.

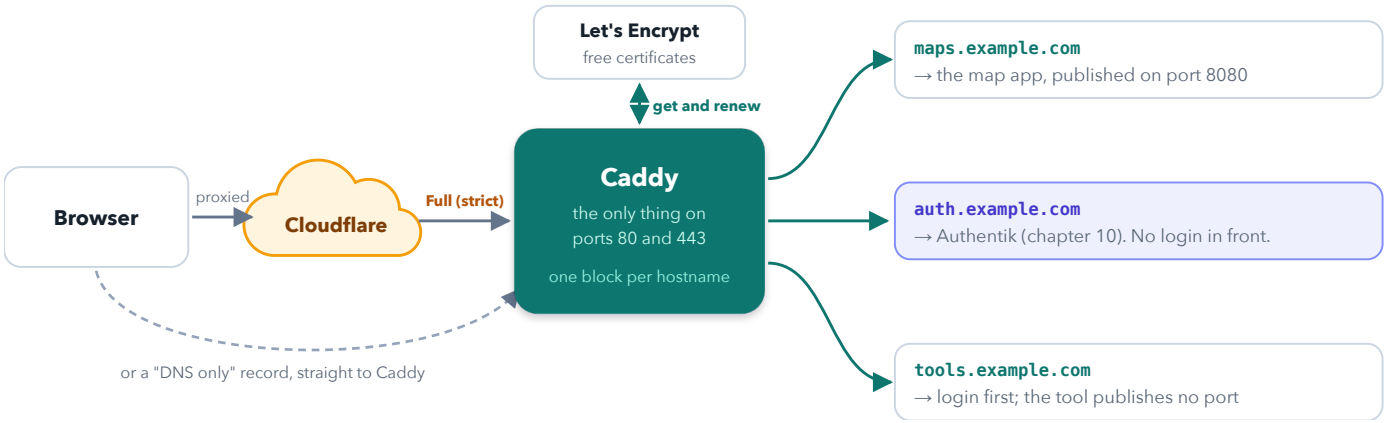
CHECK A KEY IS THERE, WITHOUT SEEING IT

```
$ scripts/lib/load-env.sh --check --file secrets/do.env DIGITALOCEAN_ACCESS_TOKEN
DIGITALOCEAN_ACCESS_TOKEN        found (file secrets/do.env, 71 chars)  # never the value
```

READ `docs/12-keys-and-scripts.md` FILES `scripts/README.md` PROMPT `14-key-safe-scripts`

9 HTTPS with Caddy

One Caddy container is the front door for every app. It gets the padlock certificates, renews them by itself, and sends each hostname to the right app.



ONE APP, START TO FINISH

1. **DNS:** an A record `maps.example.com → 203.0.113.10`.
2. **Ports:** open 80 and 443 at the firewall (and the router, if the server is at the office).
3. **The Caddyfile:**

```
maps.example.com {
    encode zstd gzip
    reverse_proxy host.docker.internal:8080
}
```

4. **Run Caddy** with a named volume on `/data`. That's where certificates live. **Keep it,** or Caddy asks for them all again and hits rate limits.
5. **Watch the log** for `certificate obtained successfully`. Renewal happens by itself from then on.

CLOUDFLARE IN FRONT

SSL MODE	CLOUDFLARE TO YOUR SERVER
Flexible	**Never** Plain HTTP. Caddy redirects to HTTPS, Cloudflare asks again over HTTP, forever.
Full	HTTPS, but any certificate is accepted.
Full (strict)	**Use this** HTTPS with a valid certificate. Caddy's is.

One wildcard certificate for `*.example.com` comes through Cloudflare's DNS. Then a new app needs no new certificate, port 80 can stay shut, and staff-only apps on an internal address still get a real padlock. It needs Caddy built with the Cloudflare plugin, and a token that can edit DNS for **one** zone only.

WHEN IT GOES WRONG

WHAT YOU SEE	WHAT IT MEANS
502 Bad Gateway from Caddy	Caddy can't reach the app: stopped, wrong port, or bound to 127.0.0.1. DNS and certificates are fine.
`ERR_TOO_MANY_REDIRECTS`	Cloudflare's SSL mode is Flexible. Switch it to Full (strict).
The challenge fails right away	DNS doesn't point at the server yet. Check with `dig +short`, wait, then reload Caddy.
Sign-in loops, no secure cookie	The app doesn't trust the proxy. Express needs `app.set('trust proxy', 1)`.

READ `docs/13-https-with-caddy.md` FILES `infra/caddy/` PROMPT `15-https-with-caddy`

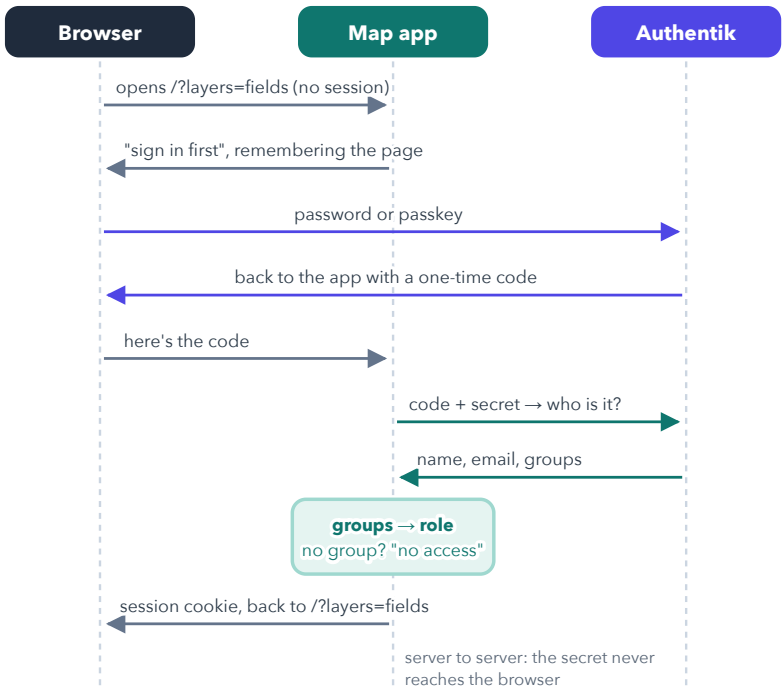
10 Optional login with Authentik

Some maps should open for anyone. Others show real data and should only open for the right people. One login server handles every app that needs it.

DO YOU NEED IT?

THE MAP IS…	LOGIN?
A public demo, a visitor map, a kiosk	No
A share link for someone outside	No The token is the lock.
Staff maps with real fields, machines or customers	Yes
A ready-made tool with no login of its own	Yes Caddy checks first.

HOW A SIGN-IN WORKS



THE WORDS

- **Identity provider (IdP):** the one server that knows your users, passwords and passkeys. We run **Authentik**: self-hosted, open source, free.
- **OIDC:** the standard the apps and the IdP use to talk.
- **Client ID and secret:** the app's own username and password *at the IdP*. The secret lives only on the app's server.
- **Redirect URI:** where the IdP sends people back. It must match exactly.
- **Groups → roles:** people go in groups (`fieldmap-operators`); the app turns groups into admin, operator or viewer.

TWO WAYS TO PROTECT A MAP APP

(a) The app signs people in itself

Login, callback, "who am I" and logout routes, about 200 lines with comments. Roles, public share pages in the same app, and clean "please sign in again" answers from the API. **Use it for your own map apps.** The demo's optional server in `example/server/` does exactly this, and it's off by default.

(b) Caddy checks with Authentik first

No code in the app at all: Caddy asks Authentik before every request ("forward auth"). Only "in the group or not". **Use it for ready-made tools** with no login of their own. One app, one mechanism: never both.

RUNNING AUTHENTIK

- **Three containers:** PostgreSQL, the server and a worker. No Redis since release 2025.10, whatever older guides say.
- **Two generated secrets,** written into `.env` unseen. Set the secret key once; never change it.
- **Set the admin password straight away.** Until you do, whoever opens the setup page first becomes the admin.
- **Bind your groups to each app.** By default every Authentik user can sign in to every application.

READ `docs/14-login-with-authentik.md` FILES `infra/authentik/` PROMPT `16-authentik-login`

10 Logout, passkeys, and failing closed

Three things that look finished long before they are.

LOGOUT THAT REALLY LOGS OUT

There are three sessions: the **app's**, **Authentik's**, and the user's sessions in **other apps**. Clearing only the app's cookie isn't a logout. The next visit goes to Authentik, Authentik still knows you, and you're straight back in without a password. To the user it looks like the button doesn't work.



✕ clearing only the first one ✓ the first two, at least

THE TRAP

Authentik's default sign-out step for new apps (`default-provider-invalidation-flow`) shows "You've logged out" and **leaves the Authentik session alive.** Send people to `default-invalidation-flow` instead, or set it as the app's invalidation flow.

The order matters: ask the server to end its session first, then go to the logout page. Don't mark yourself "signed out" in the page before leaving, or the app starts a fresh sign-in and Authentik lets you straight back in.

The test that proves it: sign out, open the app again. Authentik must ask for a password or passkey.

PASSKEYS

A passkey is a key pair kept by your phone or laptop (Face ID, Touch ID, a password manager) or a hardware key. Signing in is a glance or a fingerprint, there's nothing to type, and it can't be phished: it only answers to the real `auth.example.com`. Since every app sends people there, **one passkey works for all of them.** The demo's user menu shows "Add a passkey" when the server has the link. Ask everyone to enroll two.

FAIL CLOSED: THE SECTION NOT TO SKIP

WHAT HAPPENED TO US

A scripted redeploy replaced a production app's whole environment with an empty one. The "sign-in enabled" setting fell back to its development default of **off**, and a development shortcut ("no sign-in configured means everyone is a local admin") switched on. For a while the production map served everything to everyone, as admin. Nothing crashed, so nothing told us.

THE RULE

A missing or unsafe login setting must **stop the app from starting**. A container stuck restarting gets noticed in minutes. An app quietly running open can go unnoticed for weeks.

THREE LAYERS, AND YOU WANT ALL THREE

1. **The app refuses to start** in production without sign-in, with a weak session secret, or with a half-set login config. A public demo has to say so on purpose: `ALLOW_NO_AUTH=true`.
2. **No development shortcut in production,** even if the start-up check is somehow skipped.
3. **The deploy file refuses to deploy** without the secrets (`${SESSION_SECRET:?}`), and a lost flag defaults to the safe value.

```
$ docker run --rm react-maps-example:server
FATAL: Sign-in is off (OIDC_ENABLED is not "true")
and NODE_ENV=production. Refusing to start an open
server. …
```

CHECK FROM OUTSIDE AFTER EVERY DEPLOY

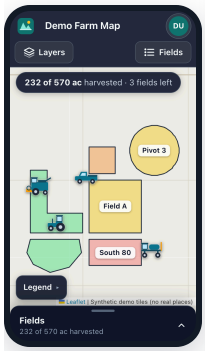
```
$ curl -s https://maps.example.com/health
{"status":"ok","auth":"oidc"}
$ curl -s -o /dev/null -w '%{http_code}\n' \
    https://maps.example.com/api/x
401     # signed out means no data
```

READ `docs/14-login-with-authentik.md` §4–6 READ `docs/07-auth.md` CODE `example/server/config.js`

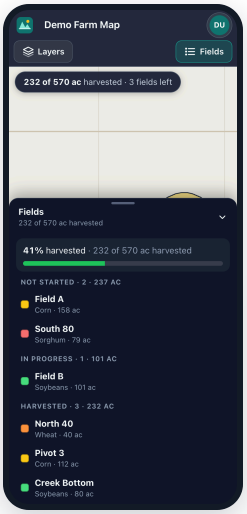
11 Mobile first

Most of our maps are used standing in a field, on a phone, in sunlight, sometimes with gloves on. Design for that first, and the desktop comes along for free.

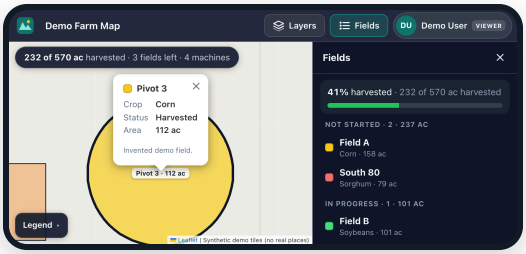
THE FOUR SIZES WE CHECK EVERY TIME · DRAWN TO THE SAME SCALE



320 × 568
The smallest phone still in use.



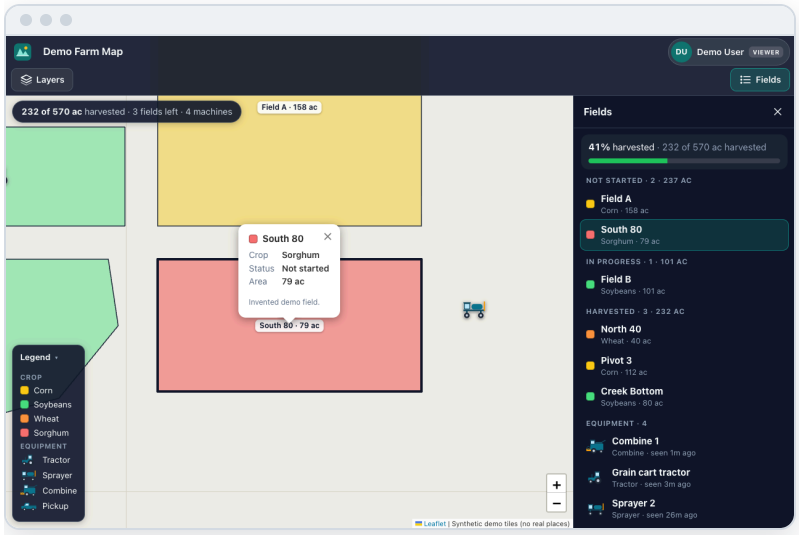
390 × 844
A typical phone, list open.



844 × 390 The same phone sideways: the one people forget. The list sheet becomes a side panel; drawers stay drawers.

LOOK FOR

- No sideways scrolling, nothing cut off.
- Nothing under the notch or the home bar.
- A top bar of one or two lines.
- The user menu opens on top.
- Buttons big enough for a thumb.



1280 × 800 A laptop. Nothing ships until every page has been looked at in all four sizes. The public share page gets its own check.

WHAT MAKES THE DIFFERENCE

- **Really full screen.** Use the "dynamic" screen height, `100dvh`. Plain `height: 100%` showed only the top quarter of the map on iPhones.
- **Tell Leaflet when its box changes:** a rotation, a drawer, the address bar sliding away. Otherwise tiles land off the grid.
- **Safe areas on every edge.** A phone held sideways has its notch on the side.
- **Every target at least 44 × 44 px,** even Leaflet's tiny popup close button.

DIALOGS BECOME BOTTOM SHEETS

On a phone, every dialog slides up from the bottom, and has three ways out: ✕, a tap outside, and Escape. Once, a typo removed a sheet's buttons and people were stuck.

SCREENSHOTS CAN LIE

A resized headless browser lays pages out wider than it claims. Test with a real phone-sized viewport (Playwright), then on a real phone, in the sun.

`READ docs/09-mobile-first.md` `PROMPT 09-mobile-pass`

12 Building it with an AI agent

The prompts say *what* to build. The docs explain *why*. An agent does its best work when it has both, and you check what it shows you.

SET UP ONCE

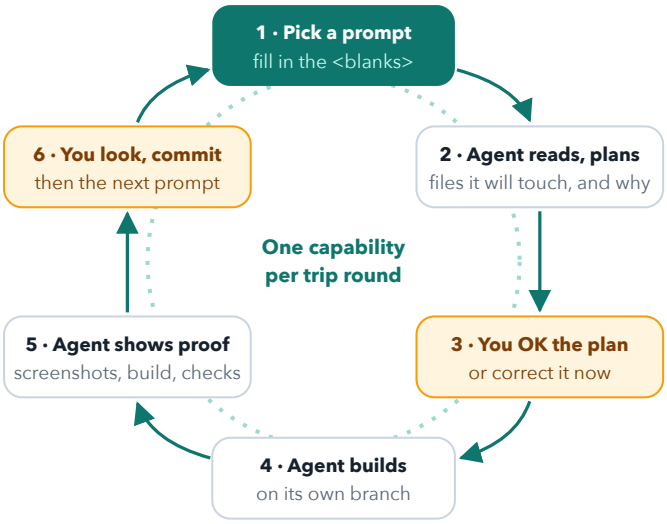
1. **Paste the house rules** from `prompts/README.md` into your agent's instructions file (`CLAUDE.md` , `AGENTS.md`). Every session then starts with them.
2. **Copy the playbook's** `docs/` into your repo as `docs/playbook/` , so the agent can read them.
3. **One prompt per branch, one branch per session.** Small steps are easier to check, and easier for the agent to get right.

EVERY PROMPT HAS THE SAME FIVE PARTS

Goal	What you want, in plain words, with your blanks filled in.
Read first	The doc sections that explain the pattern.
Constraints	The non-negotiables: what we learned the hard way.
Acceptance checks	How you both know it's finished.
How to verify	What the agent must show you: screenshots, output.

Fill in the `<angle-bracket>` parts, paste everything inside the box, and ask for the plan before any code. **Don't accept "done" until the acceptance checks pass and you've seen the screenshots yourself.**

THE LOOP



The house rules, in short

- **Nothing real in the repo:** no secrets, real hostnames, internal addresses or real data in code, docs, commits or screenshots. Placeholders only.
- **Never ask for a key in the chat.** Use, or write, a single-purpose script that reads it at run time (chapter 8).
- **Map UI rules:** menus portaled, strings escaped, overlays beside the map, the z-index ladder kept up to date.
- **Mobile first:** every page at 320, 390, 844 × 390 and 1280, with a real viewport. Targets of 44 px.
- **Definition of done:** the type check and build pass, `docker build .` passes, and screenshots at four sizes are in `outbox/` .
- **Safety:** work on a branch. Never deploy, change production or spend money without your OK in that session.

`READ prompts/README.md` `READ docs/10-lessons-learned.md`

12 The sixteen prompts

One copy-paste prompt per capability, in `prompts/`. Each names the docs to read first.

#	PROMPT	BUILDS	NEEDS
01	Start a new map app	The repo, client and server, Home and the map, satellite basemap, health and about	nothing
02	Add a layer	A switchable layer with legend, labels, tooltips and its state in the address	01
03	Slide-out panel	The layers drawer on the left, the list and details on the right	01
04	User menu that stays on top	The portaled avatar menu: About, feedback, Clear cache & reload	01
05	Custom icons	Machine clip art, tile icons, a generated app icon set	01
06	Public share link	The tiny `/s/<token>` viewer, the token list, several designs per link	01
07	Signed-in-only data	The `/m/<id>` viewer for staff, and private extras on public links	06, 11 or 16
08	GeoJSON build tool	A safe, repeatable Python tool from downloaded files to GeoJSON	nothing
09	Mobile pass	Fixes at 320, 390, landscape and desktop, with screenshots	any page
10	Dockerize + CI + Portainer	The Dockerfile, GitHub Actions to GHCR, an image-only compose file, health checks	01
11	OIDC login	The deeper sign-in: tokens in memory, refresh, back-channel logout, failing closed	01
12	Review and verify	A read-only audit: security, privacy, layering, mobile, caching, build	anything
13	Host on DigitalOcean	A droplet with firewall, reserved IP, DNS and Caddy, after a dry run you approve	10
14	Key-safe scripts	One single-purpose script per job, plus repo hygiene and the scan before every push	nothing
15	HTTPS with Caddy	The front door (or one more site on it), the Cloudflare settings, checks from outside	an app on a server
16	Login with Authentik	Authentik if you need it, then simple session sign-in (or forward auth) that fails closed	01

A NEW APP

01 → 02 → 03 → 04 → 16 → 10 → 09 → 12

Then 05, 06, 07 and 08 as you need them.

PUTTING IT ONLINE

14 → 15 or 13 → 12

Keys first, so none ever goes through the chat. 15 for your own server, 13 for a droplet. Then review the live site.

16 OR 11?

Prompt 16 is the simpler sign-in: the server keeps you signed in with a cookie, exactly like the demo's `example/server/`. Prompt 11 is the deeper version our field map uses, with tokens kept in memory. Start with 16.

TIPS THAT SAVE SESSIONS

- **Name the doc:** "Read docs/playbook/03-menus-on-top.md first" beats "make the menu stay on top".
- **Ask for the plan first** on anything big.
- **Make it show you:** screenshots, the build output, `curl /health`.

READ `prompts/README.md` PROMPT `12-review-and-verify`

12 A worked example: adding a layer

What one round of the loop looks like, using prompt 02 to add soil sample points to a map. The conversation is shortened, and the data is invented.

1 · PICK A PROMPT, FILL THE BLANKS

YOU

New branch, please: `layer-soil-samples`. Then: *Add a new map layer called "Soil samples" that shows sample points coloured by pH, from* `data/maps/soil-2026.geojson` … (the rest of prompt 02, pasted as is)

2 · THE AGENT READS, THEN PLANS

AGENT

I read `docs/playbook/02-the-map-page.md` sections 4, 6, 9 and 13. Plan:

- `layers/SoilSamplesLayer.tsx` (new): dots in their own pane, under the machines
- `shared/legends.ts`: one pH legend, used by the dots **and** the legend
- `LayersDrawer.tsx`: a switch with a purple accent bar
- Starts off; the file loads the first time it's switched on

One question: should labels show the sample ID or the pH value?

3 · YOU OK THE PLAN

YOU

The pH value. Go.

4 AND 5 · IT BUILDS, THEN SHOWS PROOF

AGENT

Done. Acceptance checks:

✓ the switch works, and Back and refresh keep it (`?layers=…,soil`)
✓ labels appear from zoom 14 and don't block taps
✓ a name containing `<b>x</b>` shows as text, not bold
✓ 20 on/off toggles leave no extra panes or layers behind

`npm run build` and `docker build .` pass. Screenshots at 320, 390, 844 × 390 and 1280 are in `outbox/`.

6 · YOU LOOK, AND YOU'RE THE ONE WHO DECIDES

YOU

At 320 wide the legend covers the My location button. Fix that, and send the 320 screenshot again.

AGENT

Fixed: the legend folds away under 360 px. New screenshot in `outbox/`. Shall I commit?

YOU

Yes. Commit, push the branch, open a pull request.

Why it went well

- **The doc was named,** so the agent followed our patterns instead of inventing new ones.
- **The plan came first,** and the question got answered before any code.
- **"Done" came with proof:** checks, a build, screenshots.
- **You looked** at the smallest screen, and caught what the checks missed.

WATCH FOR "SIMPLER"

If the agent offers to "simplify" something, check it isn't removing a pattern on purpose: the portal, the safe areas, `esc()`, the fail-closed check.

GOING IN CIRCLES?

Commit what works, start a fresh session, and paste the prompt again with a note about what's left.

NEVER

Paste a key into the chat, even "just this once". If the agent asks for one, point it at prompt 14.

`PROMPT 02-add-a-layer` `READ prompts/README.md "Tips"`

✓ If you remember nothing else

Almost every item here broke in one of our apps. The fixes are in the docs, and the prompts already ask for them.

01 **Portal every menu into** `<body>` at z 9998 / 9999. Raising the z-index alone never sticks.

02 **Fail closed.** Production refuses to start without its login settings, and secrets in compose use `${VAR:?}`.

03 **A scripted redeploy must send the settings back,** or it wipes every secret. Use `redeploy.py`.

04 **Give the agent scripts, not keys.** A key pasted into a chat is a leaked key: revoke it.

05 **Escape every string** that goes into map HTML.

06 **Check 320, 390, 844 × 390 and 1280,** with a real viewport, and on a real phone.

07 **Tell Leaflet when its box changes size.** It measures once.

08 **Never cache the page shell or** `/api/*` in a service worker of an online app. White screens and broken logouts came from there.

09 **Build the Docker image and open the app in a browser** before you call anything done.

10 **Cloudflare SSL mode is Full (strict).** Never Flexible.

11 **Attach droplet firewalls by tag,** and release reserved IPs you no longer use.

12 **No real tokens, hostnames or data in a public repo,** including its history. Scan before you push.

WHERE EVERYTHING LIVES

```
react-maps-playbook/
├── README.md       start here
├── LICENSE         MIT
├── docs/           the why, with code
│   ├── 01–10       the app itself
│   ├── 11–14       hosting, keys,
│   │               HTTPS, login
│   └── guide/      this guide's source
├── prompts/        16 copy-paste prompts
├── example/        the runnable demo
│   └── server/     optional sign-in
├── scripts/        key-safe helpers
└── infra/          Caddy and Authentik
```

TRY THE DEMO IN TWO MINUTES

```
$ cd example
$ npm install
$ npm run dev
  open http://localhost:5173, then try:
  /?color=status                      fields by harvest status
  /?layers=fields,labels,equipment,grid  all four layers
  /s/demo-7k2m9q4x8p                  the sample share link
```

Your first session with an agent

Paste the house rules into `CLAUDE.md`. Start an empty repo. Run prompt 01 and ask for the plan first. When the skeleton runs and you've seen it on your phone, commit it, and pick the next prompt.

BEFORE ANYTHING GOES PUBLIC

No secrets or `.env` files. No real share tokens. No real hostnames, internal addresses or admin URLs. No real data in code, sample files or screenshots, and nothing private baked into a public image. Scan the whole history, not just today's files, or better, start a fresh public repo and copy in only what you mean to share.

Every map, name, number and screenshot in this guide is invented demo data, drawn on a synthetic basemap with no real places. Hostnames such as `example.com` are reserved for documentation (RFC 2606), and `203.0.113.0/24` is a documentation address range, so nothing here points at a real server. Replace them with your own before you run anything. When you run the demo, its basemap is © OpenStreetMap contributors.